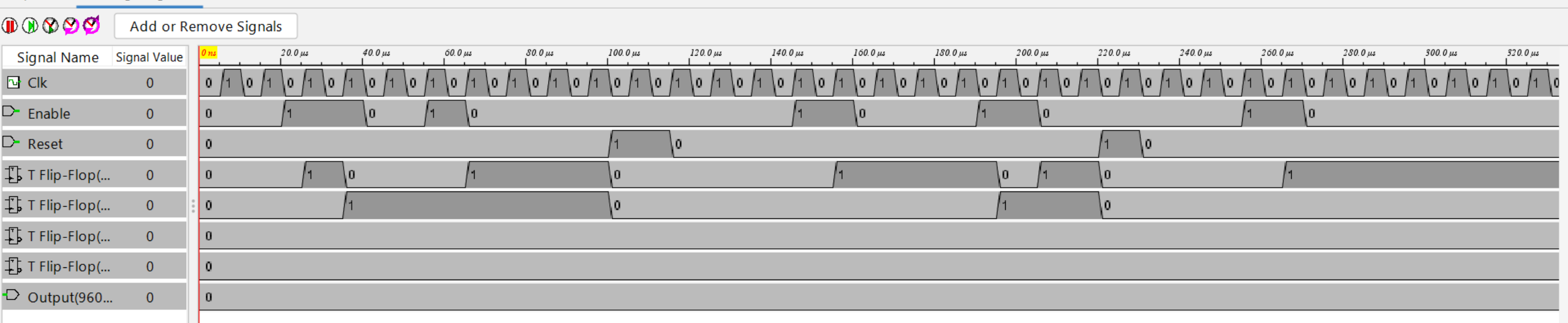
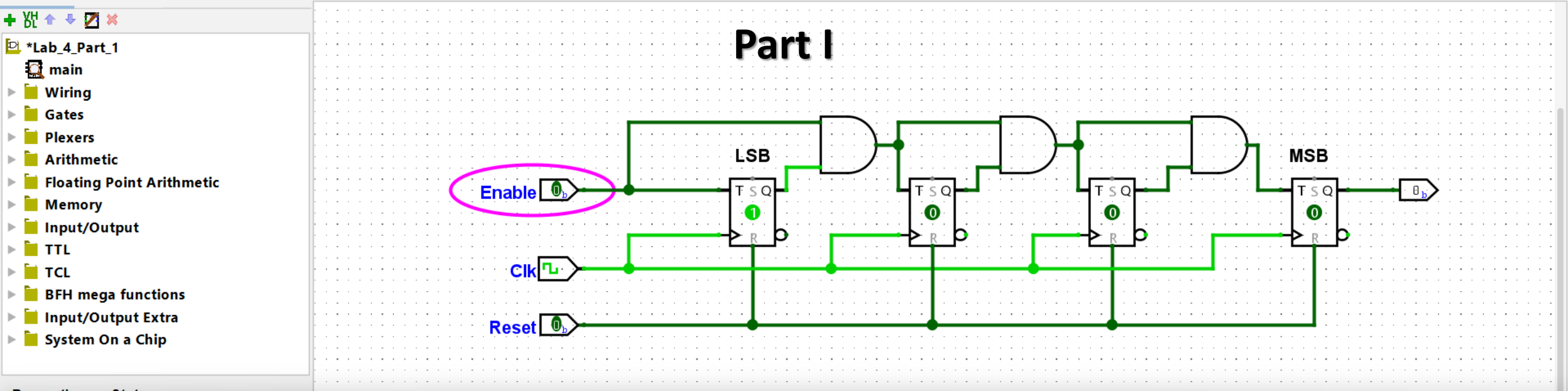
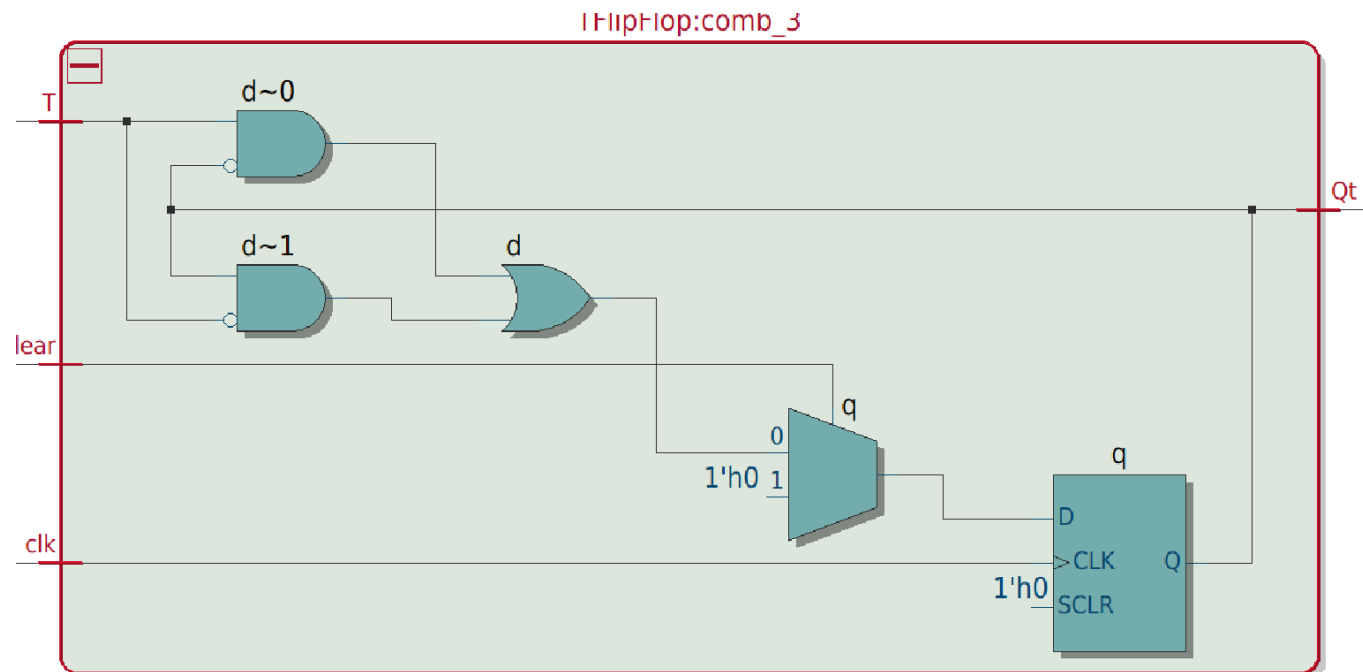
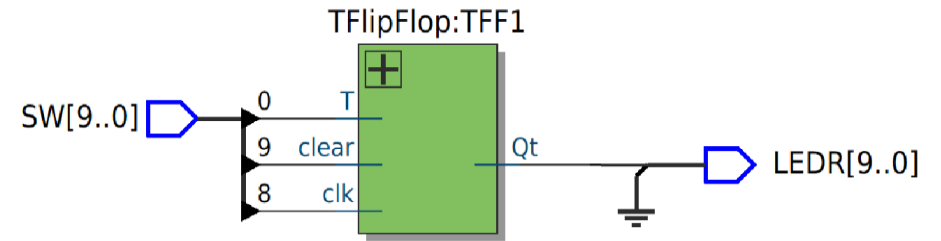




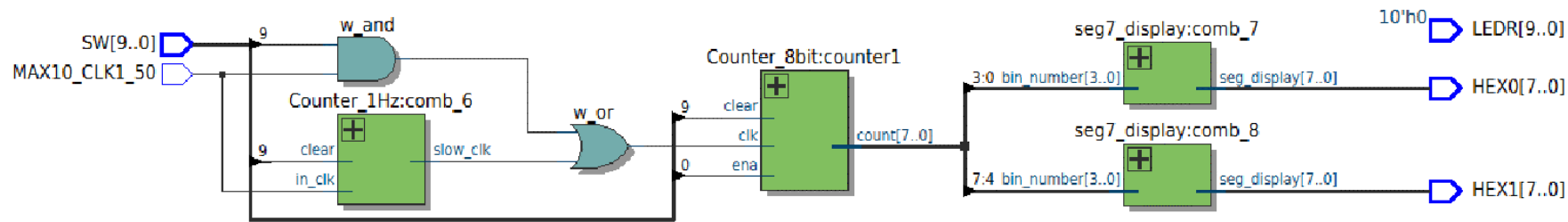
```

1  module TFlipFlop(
2      input T,
3      input clk,
4      input clear,
5      output Qt
6  );
7
8      wire d;
9      reg q;
10
11     assign d = (T & ~q) | (~T & q);
12
13     //Edge Trigger Flip Flop
14     always @(posedge clk) begin
15         if (clear) q <= 0;
16         else begin
17             q <= d;
18         end
19     end
20
21     assign Qt = q;
22
23 endmodule

```



8 bit counter with 1 Hz Clock



```

1 module Counter_1Hz(
2     input in_clk,
3     input clear,
4     output reg slow_clk
5 );
6
7     reg [25:0] clk_count;
8
9     // Clock divider: 50 MHz --> 1 Hz
10    always @(posedge in_clk, posedge clear) begin
11        if (clear) begin
12            slow_clk <= 0;
13            clk_count <= 26'd0;
14        end
15        else begin
16            if (clk_count == 26'd49_999_999) begin
17                clk_count <= 26'd0;
18                slow_clk <= 1;
19            end
20            else begin
21                clk_count <= clk_count + 1;
22                slow_clk <= 0;
23            end
24        end
25    end
26
27 endmodule

```

```

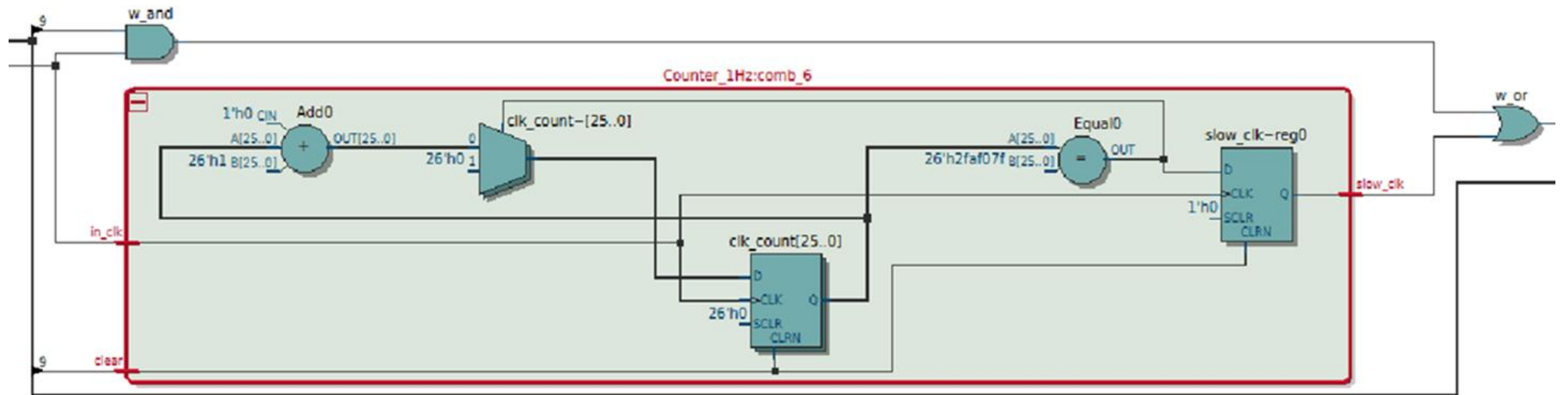
1 module Counter_8bit(
2     input ena,
3     input clk,
4     input clear,
5     output [7:0] count
6 );
7
8     wire [6:0] w_and;
9     wire [7:0] w_out;
10
11    assign count = w_out;
12
13    assign w_and[0] = w_out[0] & ena;
14    assign w_and[1] = w_out[1] & w_and[0];
15    assign w_and[2] = w_out[2] & w_and[1];
16    assign w_and[3] = w_out[3] & w_and[2];
17    assign w_and[4] = w_out[4] & w_and[3];
18    assign w_and[5] = w_out[5] & w_and[4];
19    assign w_and[6] = w_out[6] & w_and[5];
20

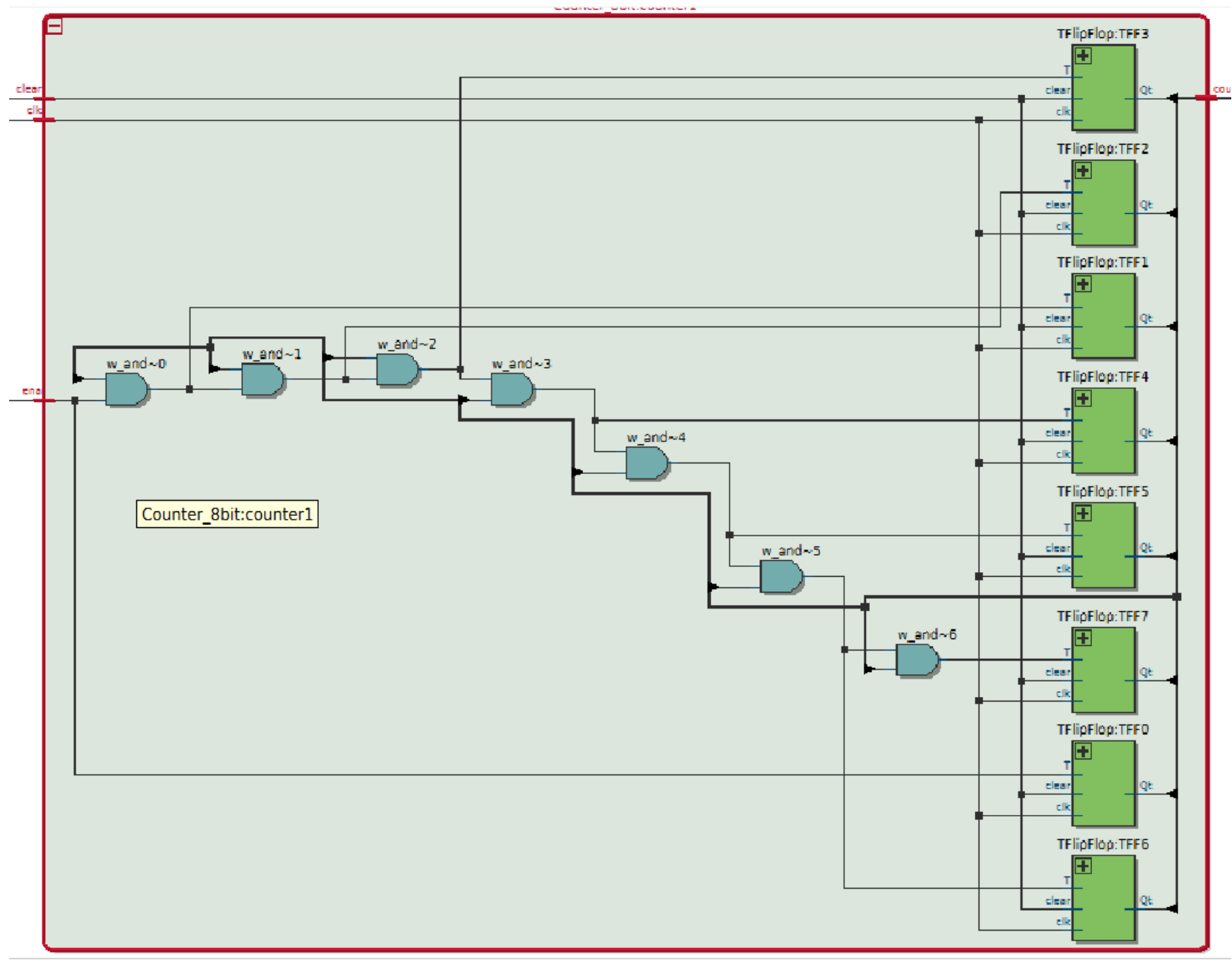
```

```

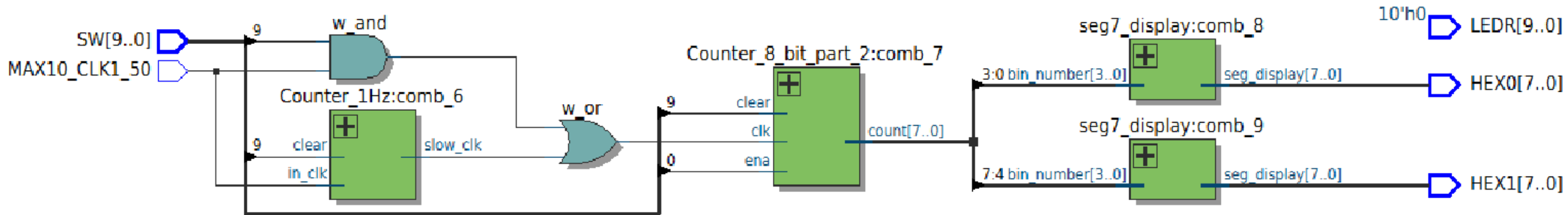
19 TFFlipFlop TFF0(
20     .T(ena),
21     .clk(clk),
22     .clear(clear),
23     .Qt(w_out[0])
24 );
25
26 TFFlipFlop TFF1(
27     .T(w_and[0]),
28     .clk(clk),
29     .clear(clear),
30     .Qt(w_out[1])
31 );
32
33 TFFlipFlop TFF2(
34     .T(w_and[1]),
35     .clk(clk),
36     .clear(clear),
37     .Qt(w_out[2])
38 );
39
40 TFFlipFlop TFF3(
41     .T(w_and[2]),
42     .clk(clk),
43     .clear(clear),
44     .Qt(w_out[3])
45 );
46
47 TFFlipFlop TFF4(
48     .T(w_and[3]),
49     .clk(clk),
50     .clear(clear),
51     .Qt(w_out[4])
52 );
53
54 TFFlipFlop TFF5(
55     .T(w_and[4]),
56     .clk(clk),
57     .clear(clear),
58     .Qt(w_out[5])
59 );
60
61 TFFlipFlop TFF6(
62     .T(w_and[5]),
63     .clk(clk),
64     .clear(clear),
65     .Qt(w_out[6])
66 );
67
68 TFFlipFlop TFF7(
69     .T(w_and[6]),
70     .clk(clk),
71     .clear(clear),
72     .Qt(w_out[7])
73 );
74
75
76

```





Part II



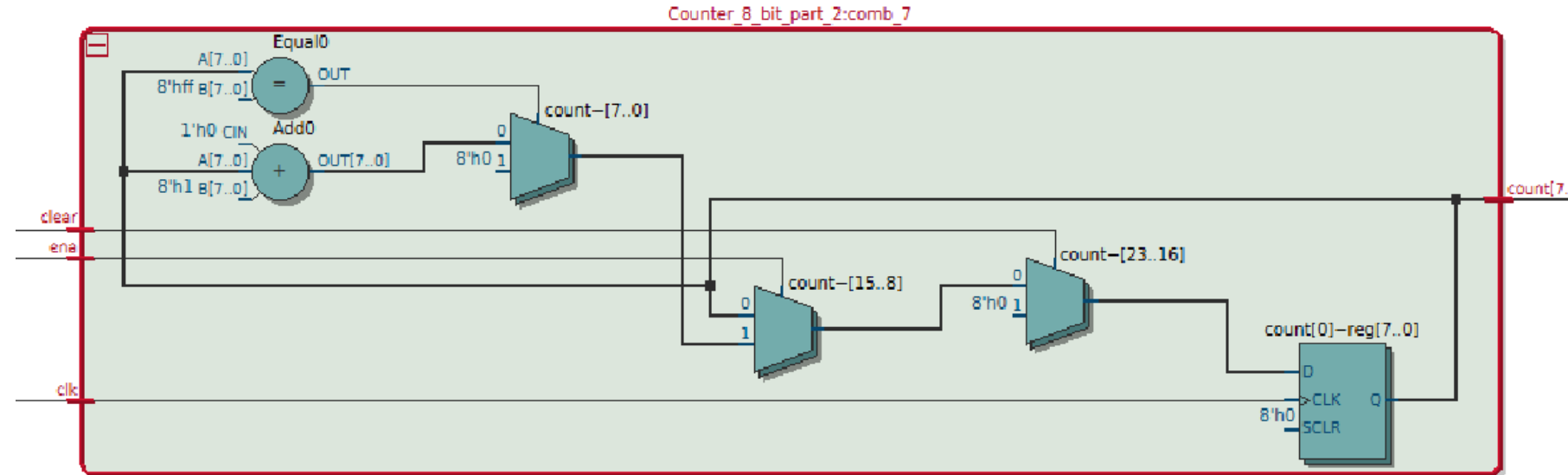
```

module Counter_8_bit_part_2(
    input ena,
    input clk,
    input clear,
    output reg [7:0] count
);

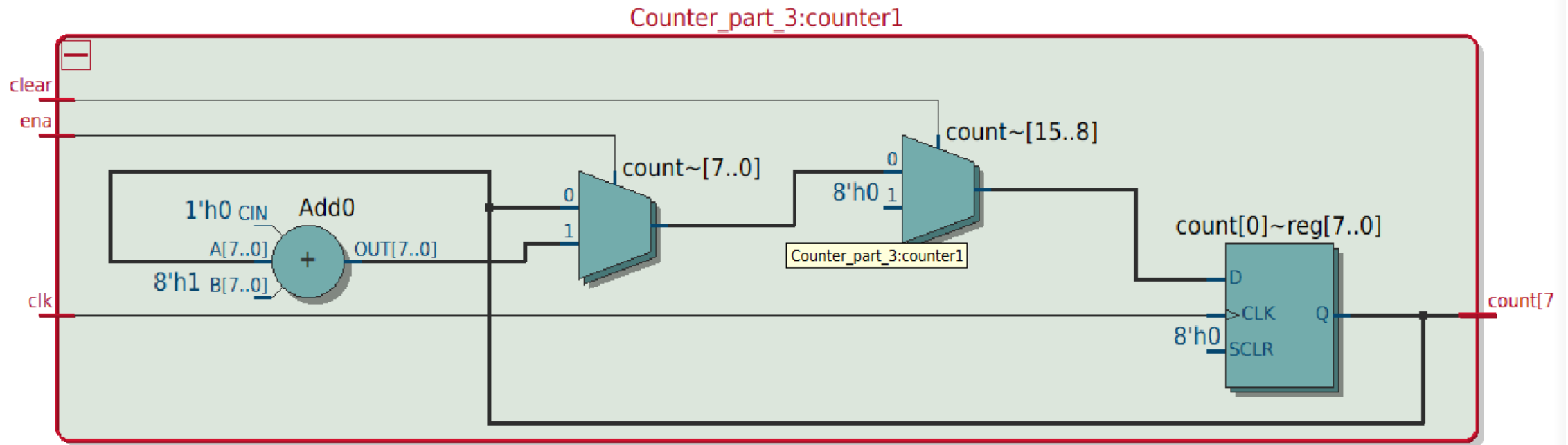
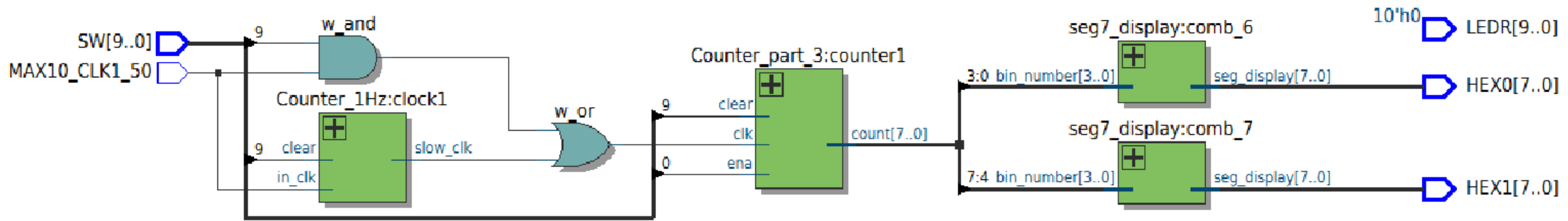
always @(posedge clk) begin
    if(clear) count <= 0;
    else begin
        if (ena) begin
            count <= count + 1;
            if (count == 8'd255) count <= 0;
        end
    end
end

endmodule

```

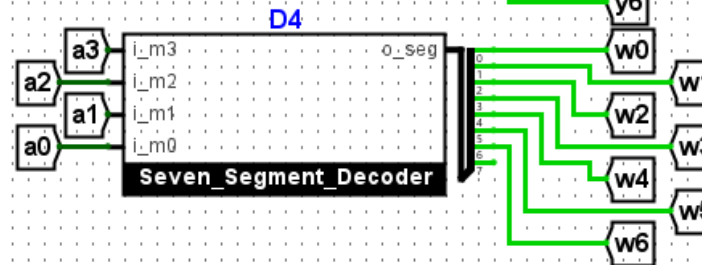
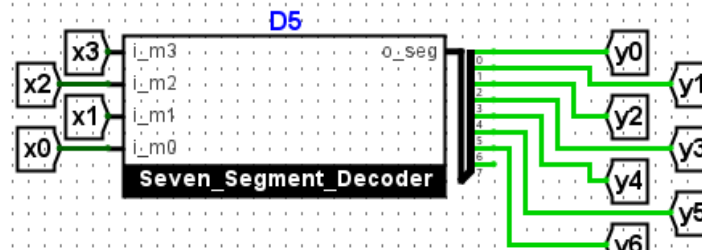
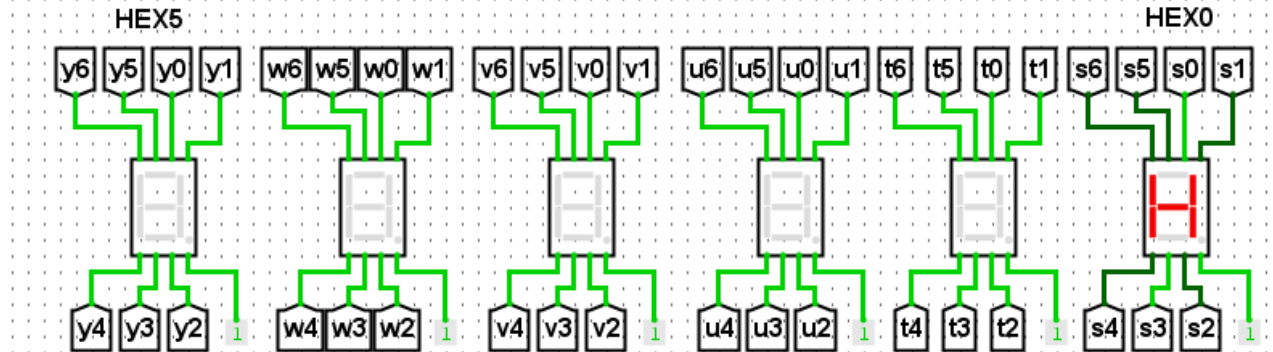
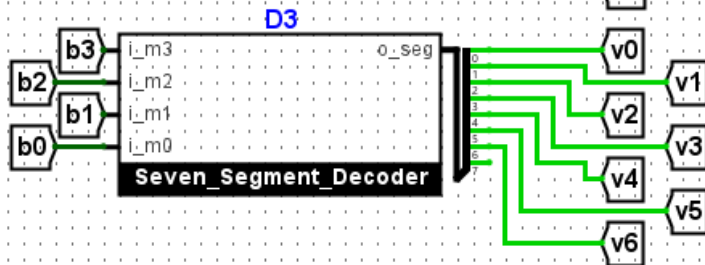
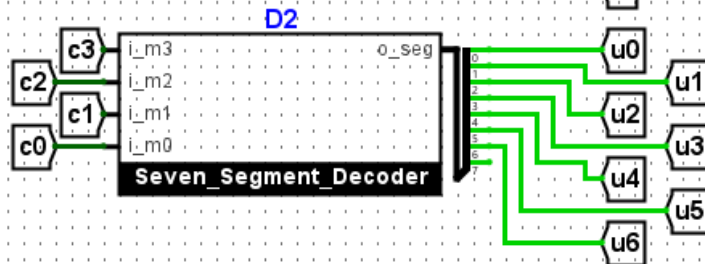
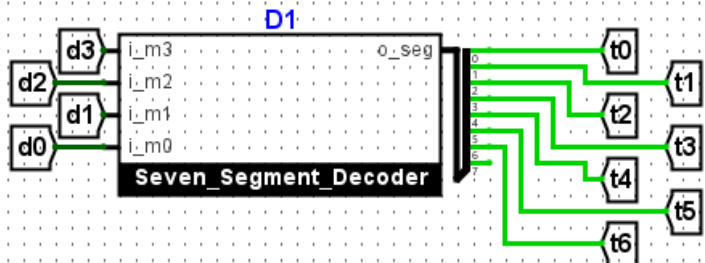
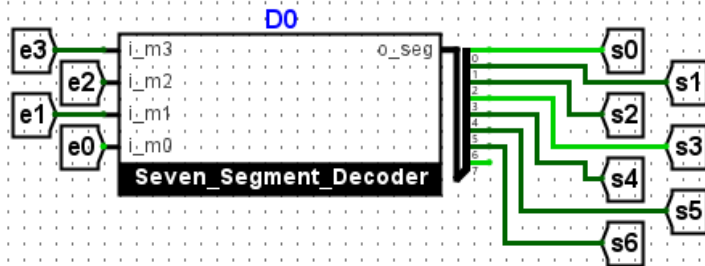
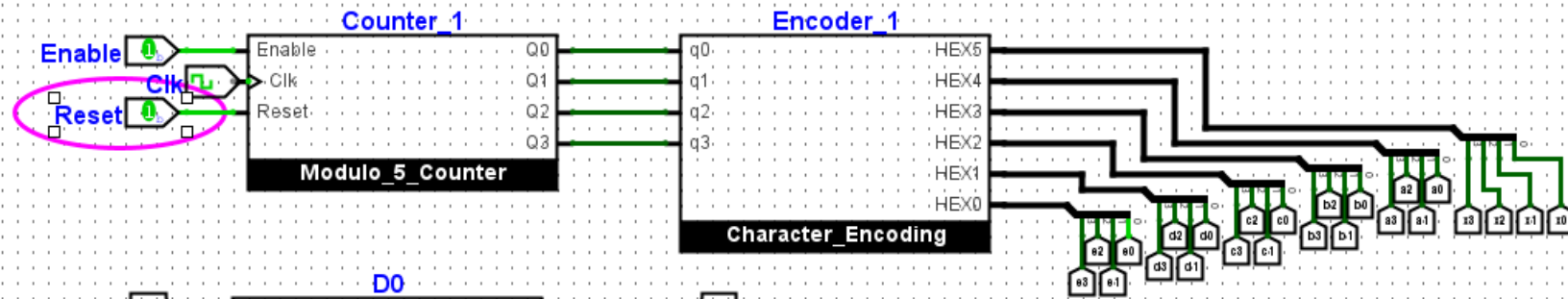


Part III

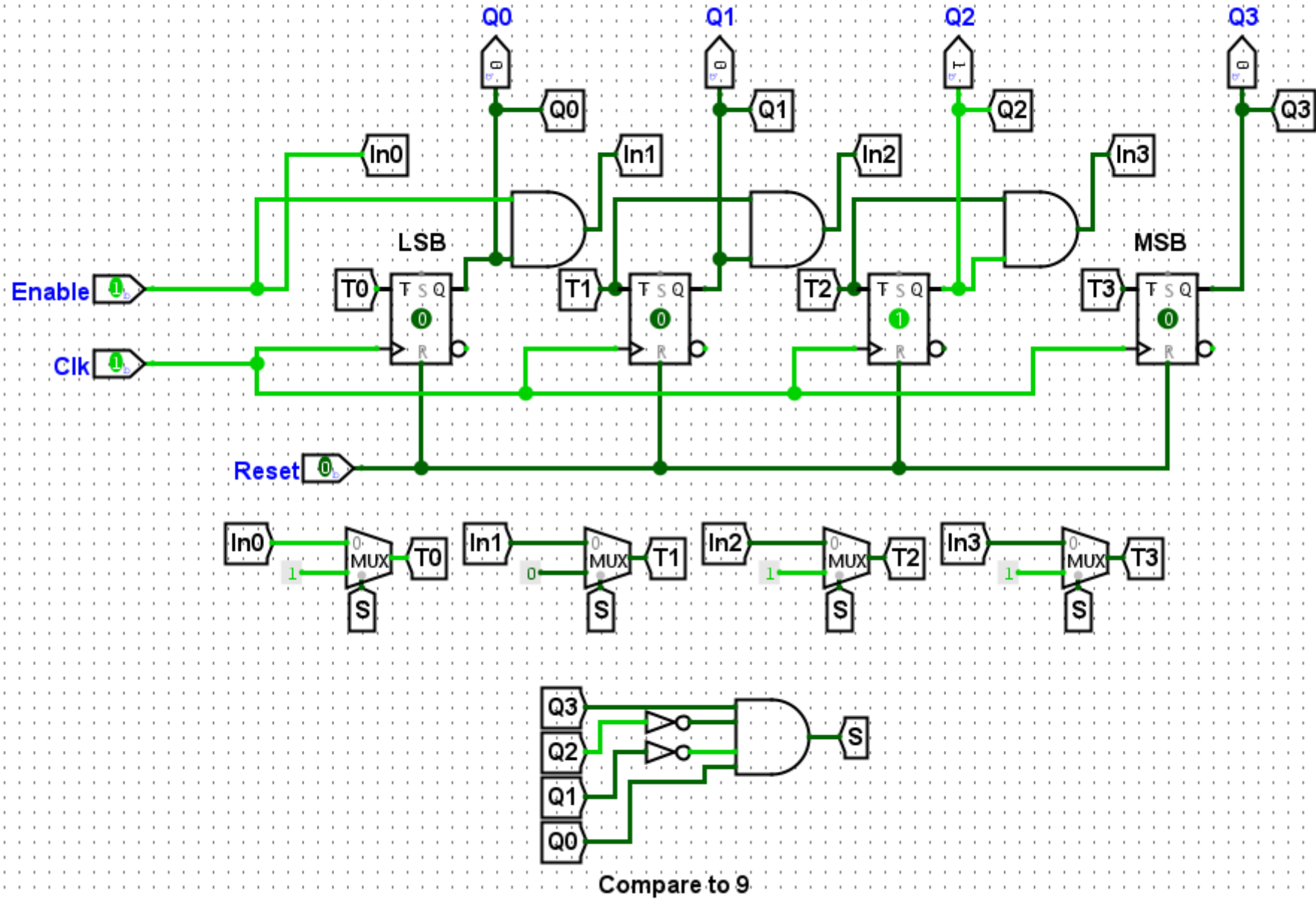


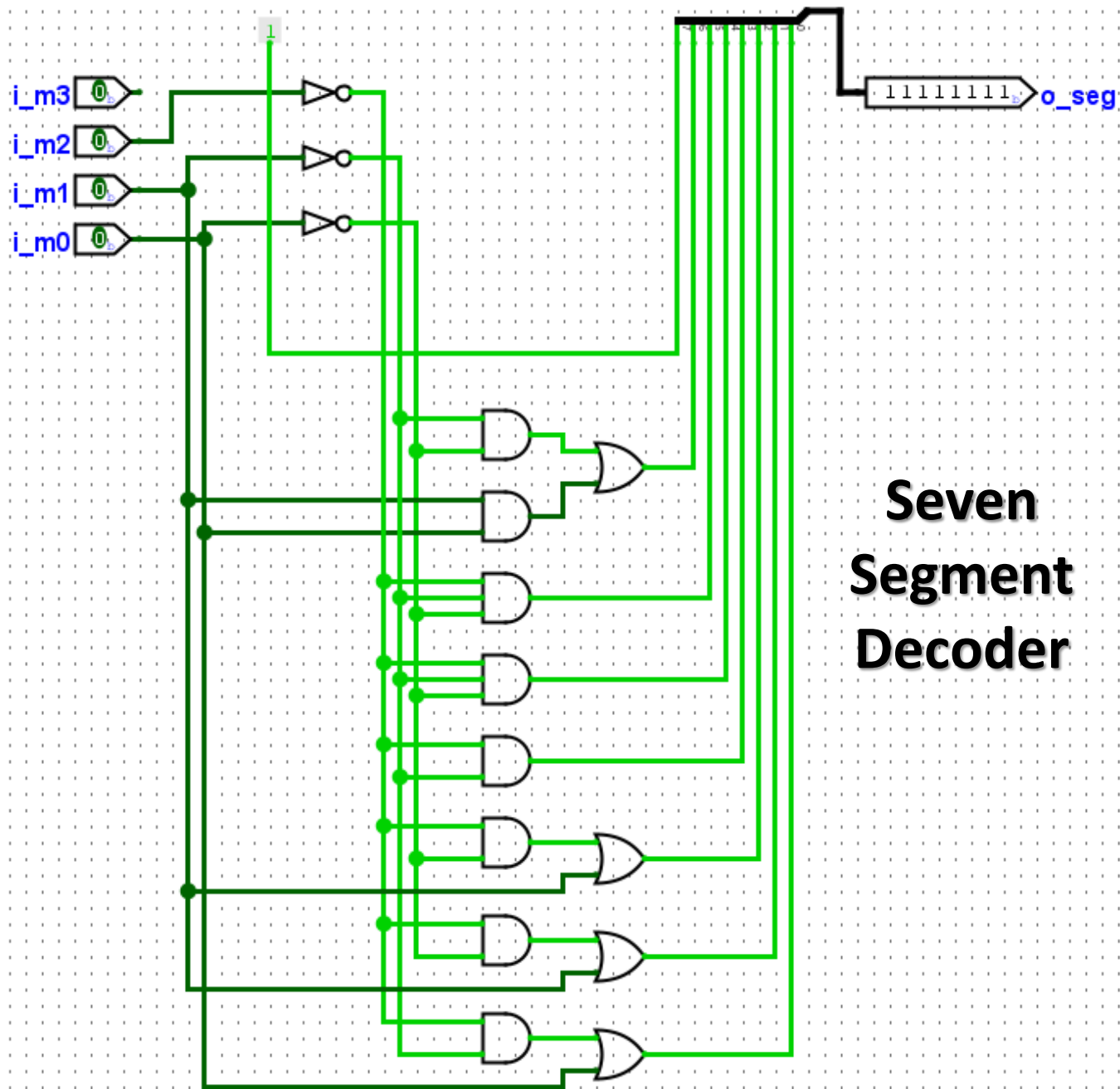
Part IV is addressed through part II and part III

Part V



Counter





Seven Segment Decoder

Output Expressions (double-click to edit):

```

o_seg7 = 1
o_seg6 =  $\overline{i_{m1}} \cdot \overline{i_{m0}} + i_{m1} \cdot i_{m0}$ 
o_seg5 =  $\overline{i_{m2}} \cdot \overline{i_{m1}} \cdot \overline{i_{m0}}$ 
o_seg4 =  $\overline{i_{m2}} \cdot \overline{i_{m1}} \cdot i_{m0}$ 
o_seg3 =  $\overline{i_{m2}} \cdot \overline{i_{m1}}$ 
o_seg2 =  $\overline{i_{m2}} \cdot i_{m0} + i_{m1}$ 
o_seg1 =  $\overline{i_{m2}} \cdot i_{m0} + i_{m1}$ 
o_seg0 =  $\overline{i_{m2}} \cdot \overline{i_{m1}} + i_{m0}$ 

```

Output Expressions (double-click to edit):

$HEX5_2 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} \cdot q0$

$HEX5_1 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} + \overline{q3} \cdot \overline{q2} \cdot q1 \cdot \overline{q0}$

$HEX5_0 = \overline{q3} \cdot \overline{q2} \cdot q0 + \overline{q3} \cdot \overline{q2} \cdot q1 \cdot \overline{q0}$

$HEX4_3 = 0$

$HEX4_2 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} \cdot q0$

$HEX4_1 = \overline{q3} \cdot \overline{q2} \cdot q0 + \overline{q3} \cdot \overline{q2} \cdot q1$

$HEX4_0 = \overline{q3} \cdot \overline{q2} \cdot q0 + \overline{q3} \cdot \overline{q2} \cdot q1$

$HEX3_3 = 0$

$HEX3_2 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} \cdot q0$

$HEX3_1 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} + \overline{q3} \cdot \overline{q2} \cdot q0$

$HEX3_0 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} \cdot q0 + \overline{q3} \cdot \overline{q2} \cdot q1 \cdot \overline{q0} + \overline{q3} \cdot \overline{q2} \cdot q1 \cdot q0 + \overline{q3} \cdot q2 \cdot \overline{q1} \cdot \overline{q0}$

$HEX2_3 = 0$

$HEX2_2 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} \cdot q0$

$HEX2_1 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} \cdot q0 + \overline{q3} \cdot \overline{q2} \cdot q1 \cdot \overline{q0} + \overline{q3} \cdot \overline{q2} \cdot q1 \cdot q0$

$HEX2_0 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} \cdot q0 + \overline{q3} \cdot \overline{q2} \cdot q1 \cdot \overline{q0} + \overline{q3} \cdot \overline{q2} \cdot q1 \cdot q0$

$HEX1_3 = 0$

$HEX1_2 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} \cdot q0$

$HEX1_1 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} + \overline{q3} \cdot \overline{q2} \cdot q1 \cdot \overline{q0} + \overline{q3} \cdot \overline{q2} \cdot q1$

$HEX1_0 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} \cdot q0 + \overline{q3} \cdot q1 \cdot \overline{q0} + \overline{q2} \cdot \overline{q1} \cdot \overline{q0}$

$HEX0_3 = 0$

$HEX0_2 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} \cdot q0$

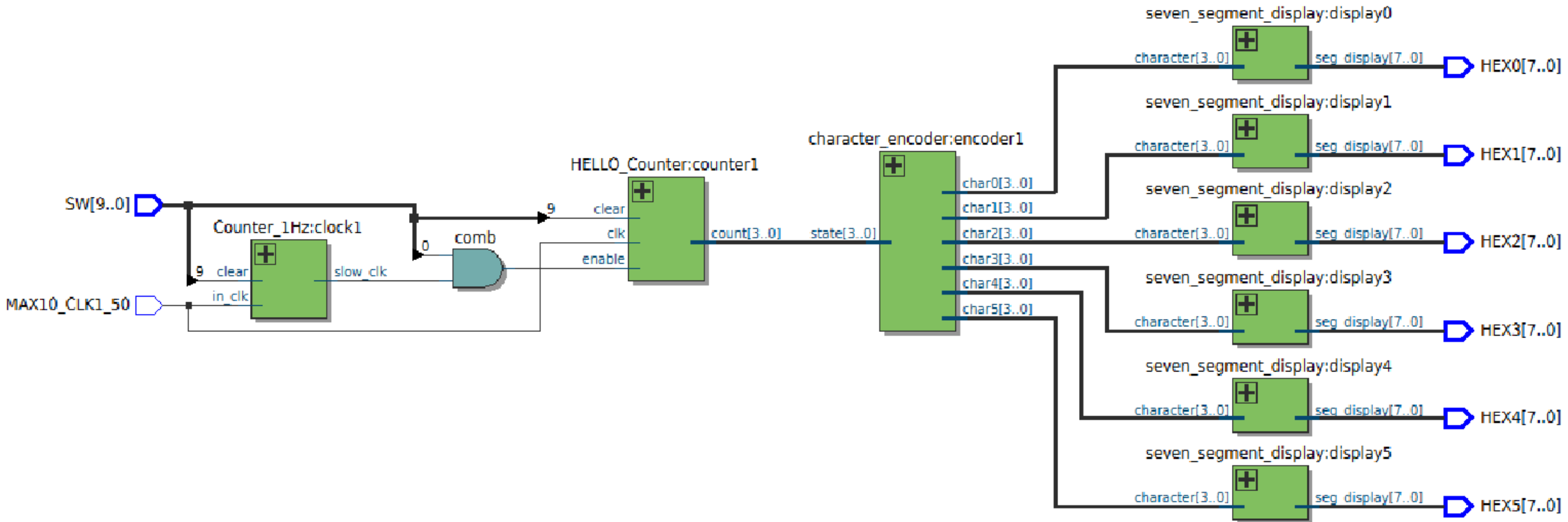
$HEX0_1 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} + \overline{q3} \cdot q1 \cdot \overline{q0} + \overline{q3} \cdot \overline{q2} \cdot q1 + \overline{q3} \cdot \overline{q2} \cdot q0$

$HEX0_0 = \overline{q3} \cdot \overline{q2} \cdot \overline{q1} + \overline{q3} \cdot q1 \cdot \overline{q0} + \overline{q3} \cdot \overline{q2} \cdot q1 + \overline{q3} \cdot \overline{q2} \cdot q0$

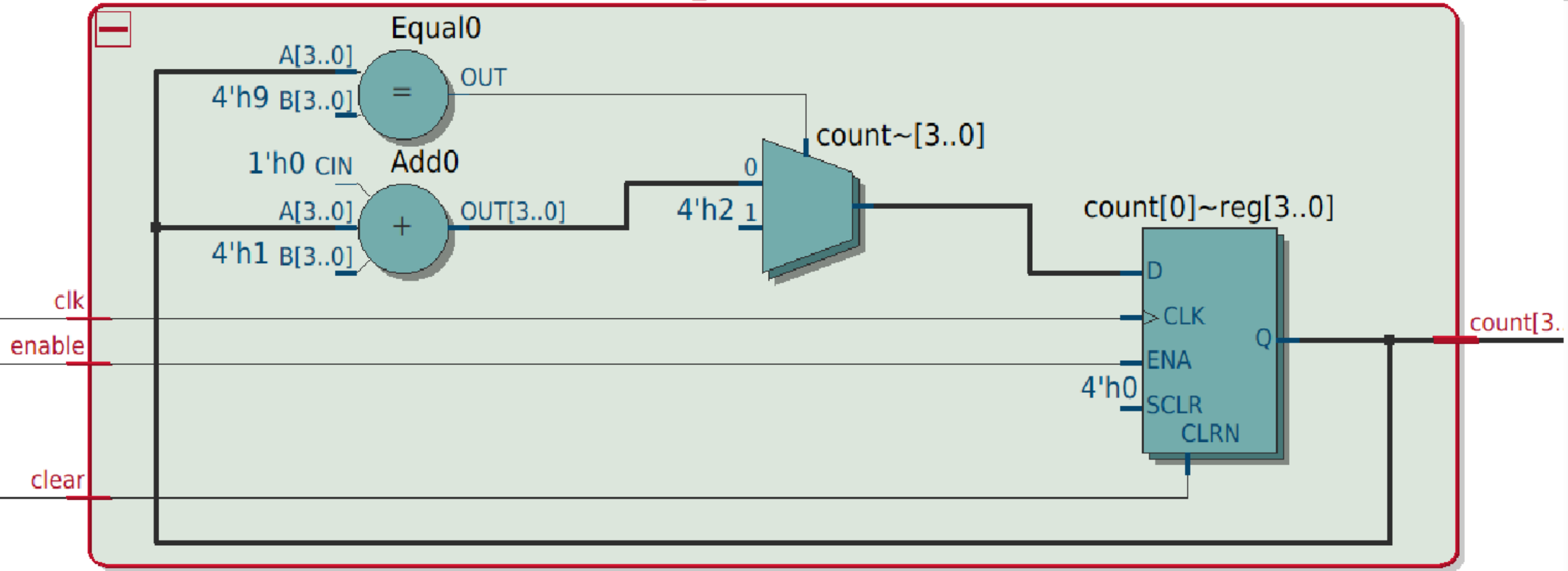
Character Encoder

q0	q1	q2	q3	HEX5[3..0]	HEX4[3..0]	HEX3[3..0]	HEX2[3..0]	HEX1[3..0]	HEX0[3..0]
0	0	0	0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 1
0	0	0	1	0 0 1 1	0 1 0 0	0 0 0 0	0 0 0 1	0 0 1 0	0 0 1 1
0	0	1	0	0 0 0 0	0 0 0 1	0 0 1 0	0 0 1 1	0 0 1 1	0 1 0 0
0	0	1	1	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0
0	1	0	0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 1	0 0 1 0	0 0 1 1
0	1	0	1	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0
0	1	1	0	0 0 1 0	0 0 1 1	0 0 1 1	0 1 0 0	0 0 0 0	0 0 0 1
0	1	1	1	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0
1	0	0	0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 1	0 0 1 0
1	0	0	1	0 1 0 0	0 0 0 0	0 0 0 1	0 0 1 0	0 0 1 1	0 0 1 1
1	0	1	0	0 0 0 1	0 0 1 0	0 0 1 1	0 0 1 1	0 1 0 0	0 0 0 0
1	0	1	1	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0
1	1	0	0	0 0 0 0	0 0 0 0	0 0 0 1	0 0 1 0	0 0 1 1	0 0 1 1
1	1	0	1	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0
1	1	1	0	0 0 1 1	0 0 1 1	0 1 0 0	0 0 0 0	0 0 0 1	0 0 1 0
1	1	1	1	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0

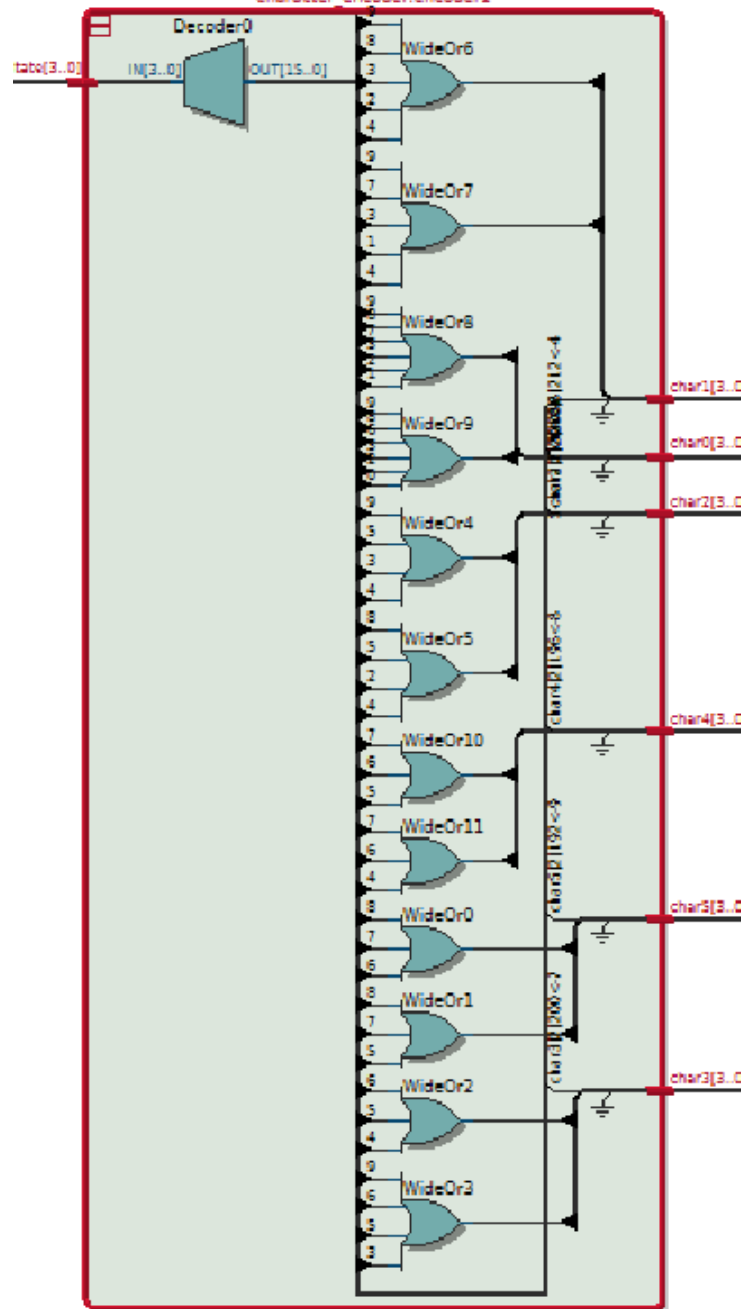
RTL Schematic



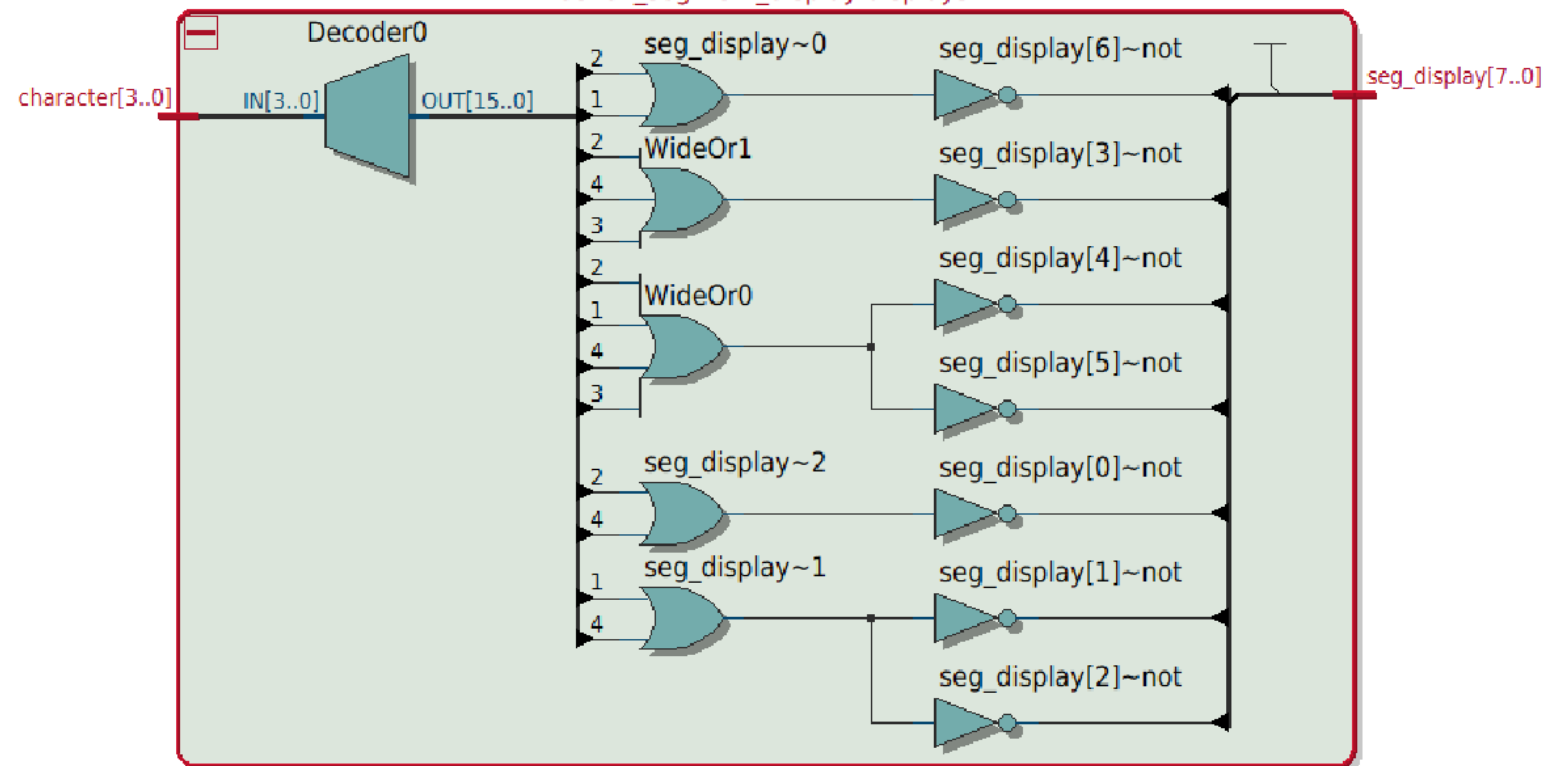
HELLO_Counter:counter1



character_encoder:encoder1



seven_segment_display:display5



```

module HELLO_Counter(
    input      clk,
    input      enable,
    input      clear,
    output reg [3:0] count
);

    always @(posedge clk, posedge clear) begin

        if (clear)
            count <= 4'd0;

        else if (enable) begin

            if (count == 4'd9)
                count <= 4'd4;
            else
                count <= count + 1'b1;

        end

    end

endmodule

```

```

module seven_segment_display(
    input      [3:0] character,
    output reg [7:0] seg_display
);

    always @(*) begin
        case(character)

            4'b0000: seg_display = 8'b11111111; // Blank
            4'b0001: seg_display = 8'b10001001; // H
            4'b0010: seg_display = 8'b10000110; // E
            4'b0011: seg_display = 8'b11000111; // L
            4'b0100: seg_display = 8'b11000000; // 0

            default: seg_display = 8'b11111111; // Blank

        endcase
    end

endmodule

```

```

1  module character_encoder(
2      input      [3:0] state,
3
4      output reg [3:0] char5,
5      output reg [3:0] char4,
6      output reg [3:0] char3,
7      output reg [3:0] char2,
8      output reg [3:0] char1,
9      output reg [3:0] char0
10 );
11
12 // Character codes
13 localparam BLANK = 4'b0000;
14 localparam H    = 4'b0001;
15 localparam E    = 4'b0010;
16 localparam L    = 4'b0011;
17 localparam O    = 4'b0100;
18
19 always @(*) begin
20
21     // Default: all displays
22     char5 = BLANK;

```

```

22     char5 = BLANK;
23     char4 = BLANK;
24     char3 = BLANK;
25     char2 = BLANK;
26     char1 = BLANK;
27     char0 = BLANK;
28
29     case (state)
30
31         // _ _ _ _ H
32         4'd0: begin
33             char0 = H;
34         end
35
36         // _ _ _ _ H E
37         4'd1: begin
38             char1 = H;
39             char0 = E;
40         end
41
42         // _ _ _ H E L
43         4'd2: begin
44             char2 = H;
45             char1 = E;
46             char0 = L;
47     end

```

```

49         // _ _ H E L L
50         4'd3: begin
51             char3 = H;
52             char2 = E;
53             char1 = L;
54             char0 = L;
55         end
56
57         // _ H E L L O
58         4'd4: begin
59             char4 = H;
60             char3 = E;
61             char2 = L;
62             char1 = L;
63             char0 = O;
64         end
65
66         // H E L L O _
67         4'd5: begin
68             char5 = H;
69             char4 = E;
70             char3 = L;
71             char2 = L;
72             char1 = O;
73         end
74     end

```

```

76 4'd6: begin
77     char5 = E;
78     char4 = L;
79     char3 = L;
80     char2 = 0;
81     char0 = H;
82 end
83
84 // L L 0 _ H E
85 4'd7: begin
86     char5 = L;
87     char4 = L;
88     char3 = 0;
89     char1 = H;
90     char0 = E;
91 end
92
93 // L 0 _ H E L
94 4'd8: begin
95     char5 = L;
96     char4 = 0;
97     char2 = H;
98     char1 = E;
99     char0 = L;
100 end
101

```

```

101
102 // 0 _ H E L L
103 4'd9: begin
104     char5 = 0;
105     char3 = H;
106     char2 = E;
107     char1 = L;
108     char0 = L;
109 end
110
111 default: begin
112     // Defaults above keep all displays blank
113 end
114
115 endcase
116
117 end
118 endmodule

```

```

1  module main(
2      input      MAX10_CLK1_50,
3      input  [9:0] SW,
4
5      output [7:0] HEX0,
6      output [7:0] HEX1,
7      output [7:0] HEX2,
8      output [7:0] HEX3,
9      output [7:0] HEX4,
10     output [7:0] HEX5
11 );
12
13     wire clear;
14     wire slow_clk;
15     wire [3:0] count;
16
17     wire [3:0] char0;
18     wire [3:0] char1;
19     wire [3:0] char2;
20     wire [3:0] char3;
21     wire [3:0] char4;
22     wire [3:0] char5;
23
24     assign clear = SW[9];
25
26

```

```

28     // 1 Hz timing pulse
29     // -----
30     Counter_1Hz clock1 (
31         .in_clk   (MAX10_CLK1_50),
32         .clear    (clear),
33         .slow_clk (slow_clk)
34     );
35
36
37     // -----
38     // HELLO state counter
39     // 0,1,2,...,9,4,5,...,9,4,...
40     // -----
41     HELLO_Counter counter1 (
42         .clk      (MAX10_CLK1_50),
43         .enable   (slow_clk & SW[0]),
44         .clear    (clear),
45         .count    (count)
46     );
47
48
49     // -----
50     // Character encoder
51     // -----
52     character_encoder encoder1 (
53         state (count)

```

```

55         .char5 (char5),
56         .char4 (char4),
57         .char3 (char3),
58         .char2 (char2),
59         .char1 (char1),
60         .char0 (char0)
61     );
62
63     // -----
64     // Seven-segment decoders
65     // -----
66     seven_segment_display display5 (
67         .character (char5),
68         .seg_display (HEX5)
69     );
70
71     seven_segment_display display4 (
72         .character (char4),
73         .seg_display (HEX4)
74     );
75
76     seven_segment_display display3 (
77         .character (char3),
78         .seg_display (HEX3)
79     );
80

```

```

81
82     seven_segment_display display2 (
83         .character (char2),
84         .seg_display (HEX2)
85     );
86
87     seven_segment_display display1 (
88         .character (char1),
89         .seg_display (HEX1)
90     );
91
92     seven_segment_display display0 (
93         .character (char0),
94         .seg_display (HEX0)
95     );
96
97     endmodule

```